

η_t - learning rate by t

• $\eta_t = \eta_0$ - constant scheduler

• $\eta_t = \begin{cases} \eta_0 & 0 < t \leq t_0 \\ \eta_1 & t_0 < t \leq t_1 \end{cases}$ - step LR scheduler

$$\eta_0 > \eta_1 > \dots$$

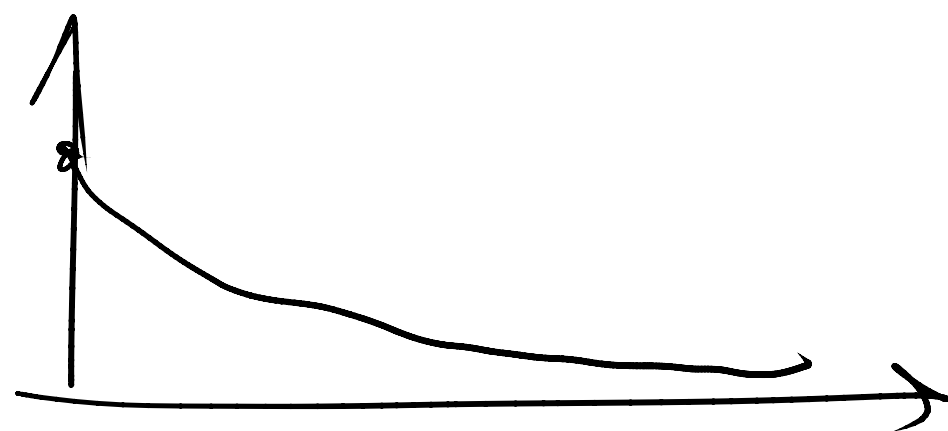
• $\eta_t = \begin{cases} \eta_0 & 0 < t \leq t_0 \\ \eta_1 & t_0 < t \leq t_1 \end{cases}$ $Q_{\text{valid}}^{t_j} \leq Q_{\text{valid}}^{t_{j-i}}$
 $i \in [1, n]$

$$\eta_j = \gamma \eta_{j-1} \quad \gamma < 1 \quad n - \text{epoch without change}$$

Reduce LR on Plateau

• $\eta_t = \gamma \eta_{t-1} \quad \gamma < 1$

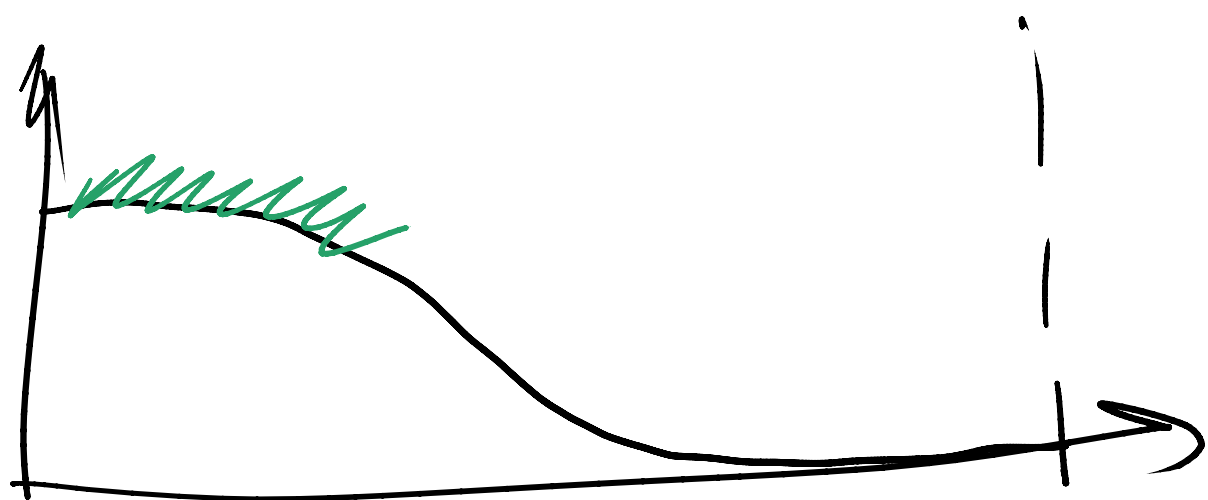
Exponential LR



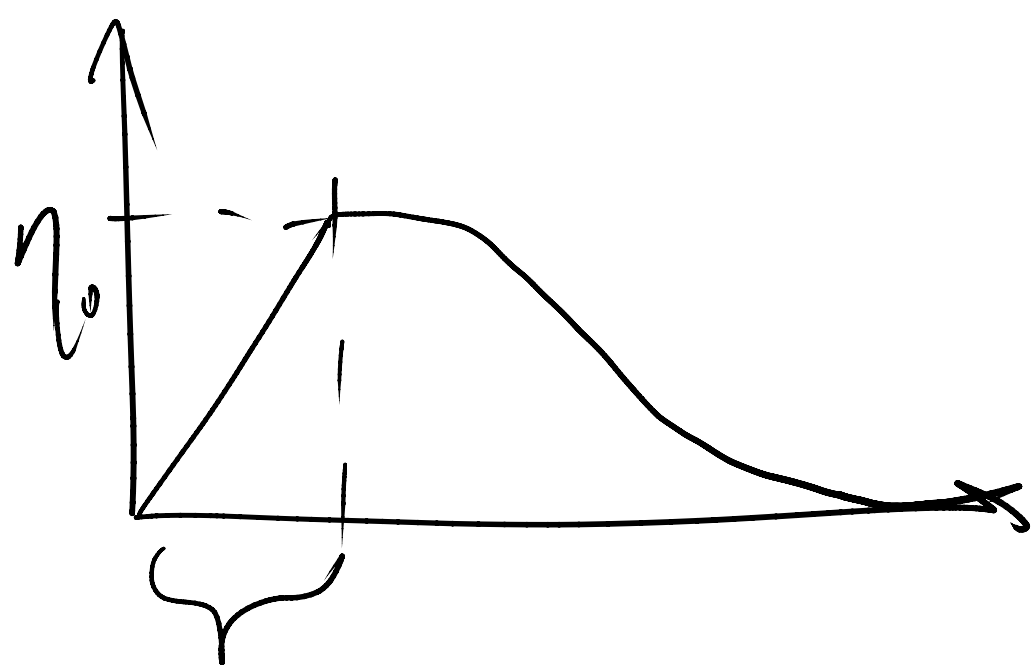
$$\eta_t = \eta_0 \cdot \frac{1}{2} \left(1 + \cos \left(\frac{\pi t}{T} \right) \right)$$

Cosine LR

Cosine Annealing



η_0, T

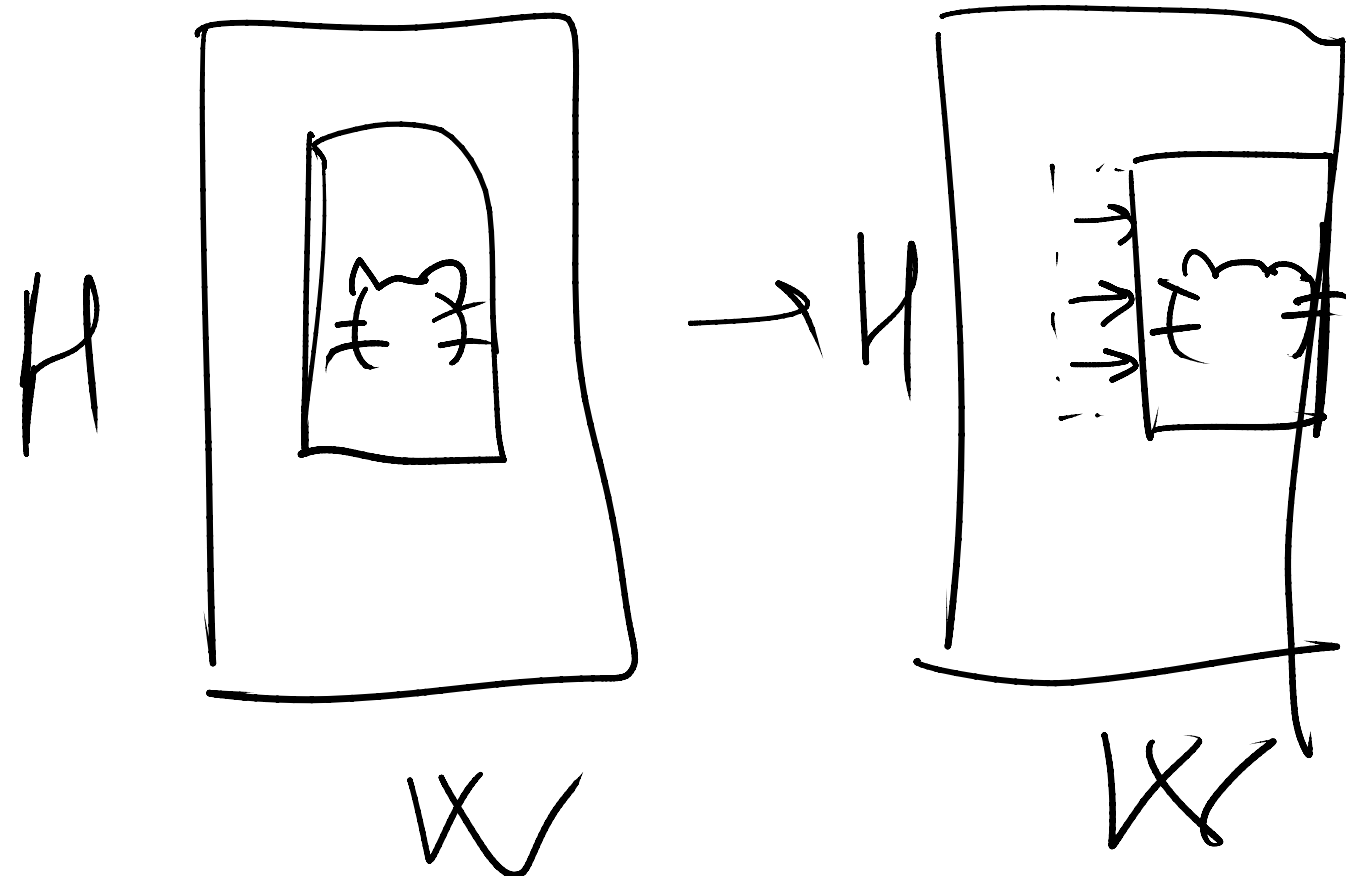


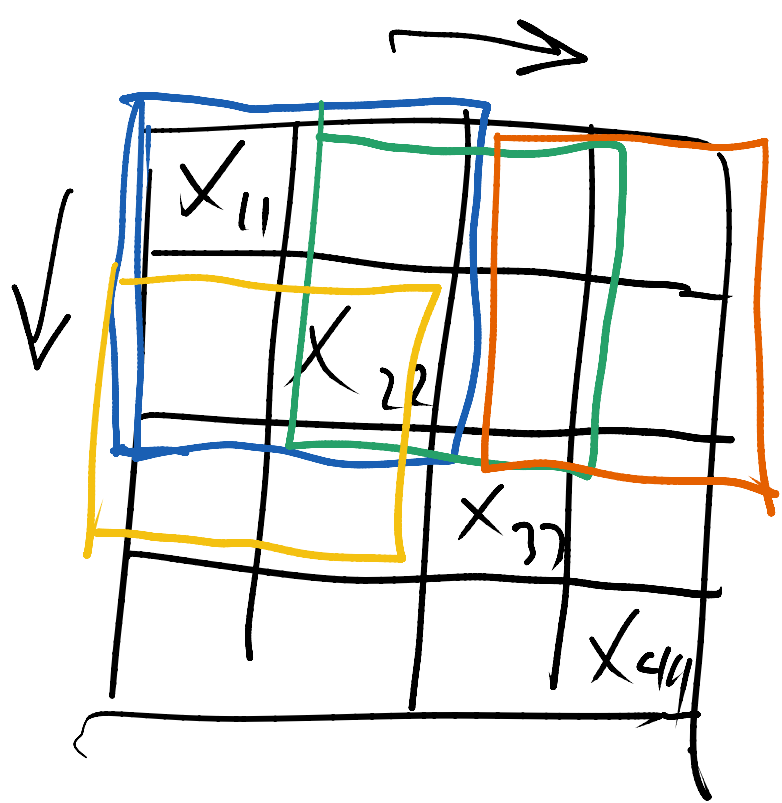
Warmup LR

first few epochs

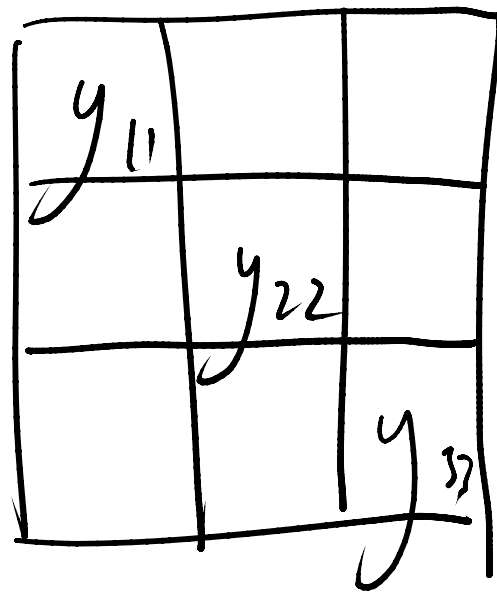
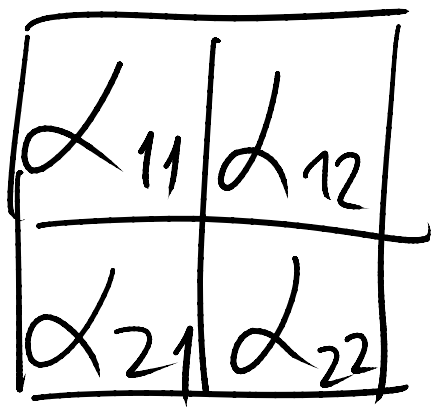
Inductive bias

$H \times W \times 3$





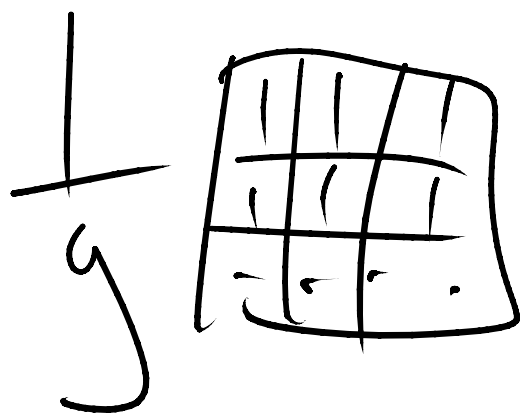
learnable



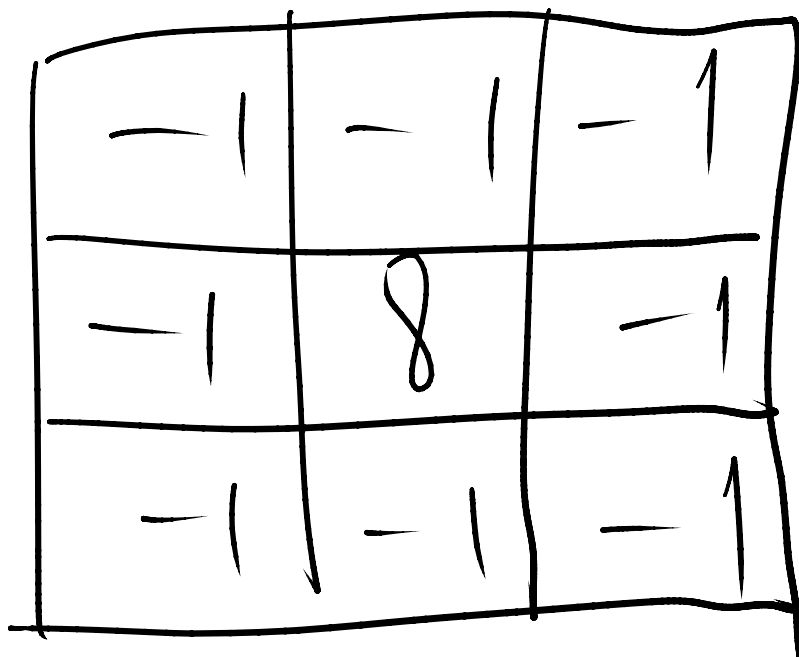
filter

$$y_{11} = \sum_{i,j \in \square} \alpha_{ij} x_{ij}$$

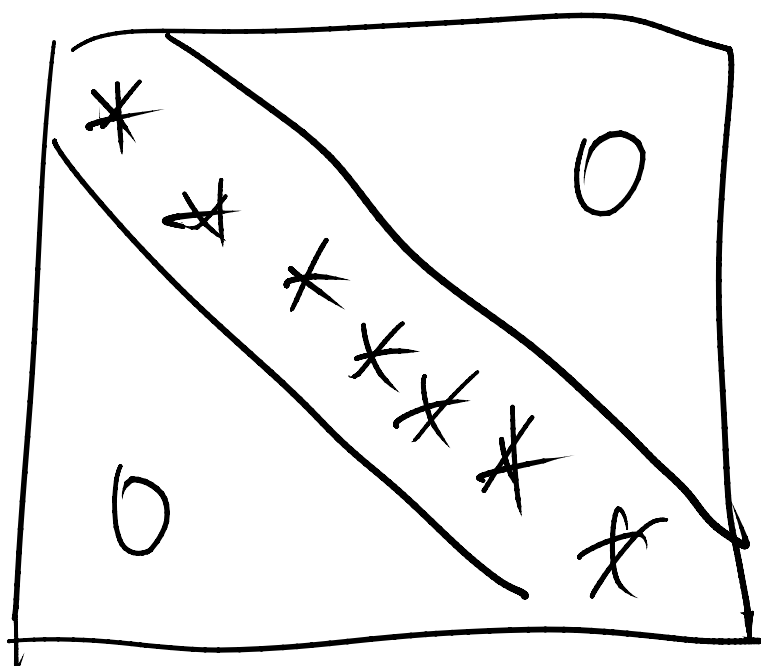
$$y_{12} = \sum_{i,j \in \square} \alpha_{ij} x_{ij}$$



$\frac{1}{8}$



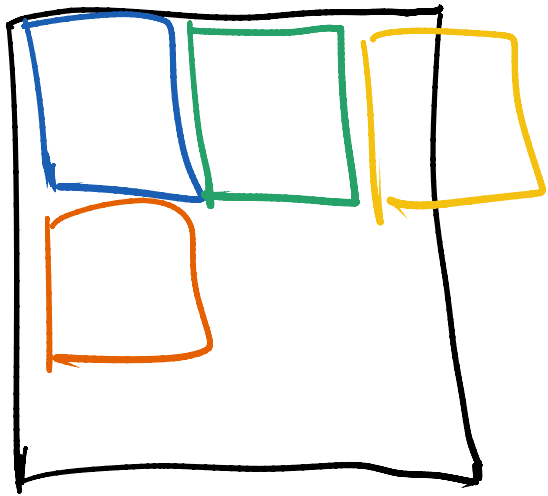
Convolutional Neural Networks



$$\begin{bmatrix} x_{11} \\ x_{12} \\ \vdots \end{bmatrix} = \begin{bmatrix} y_{11} \\ y_{12} \\ \vdots \end{bmatrix}$$

Max Pooling

2x2



$$\max_{ij} x_{ij}$$

A single blue square representing the output of the max pooling operation.

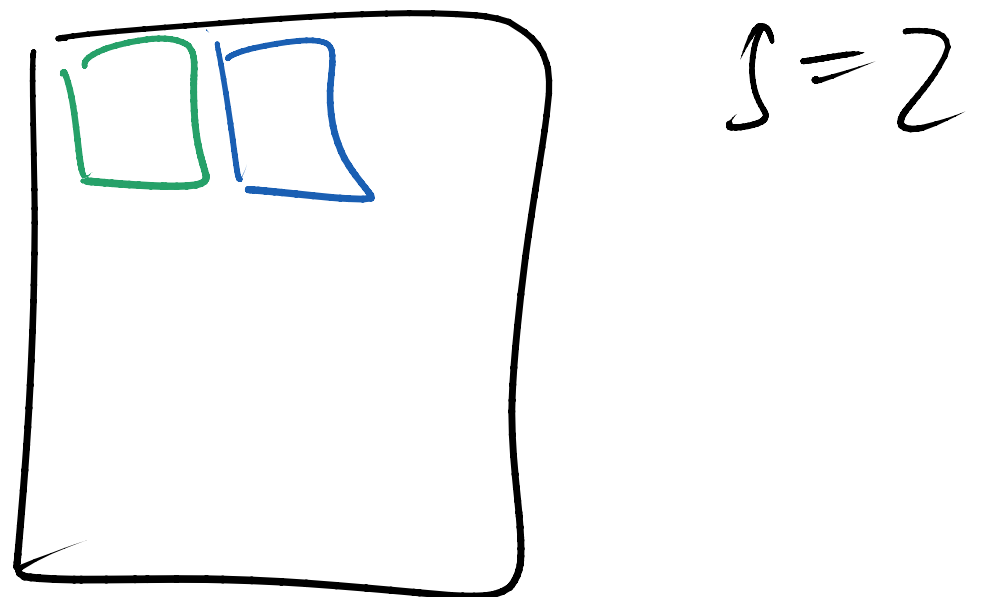
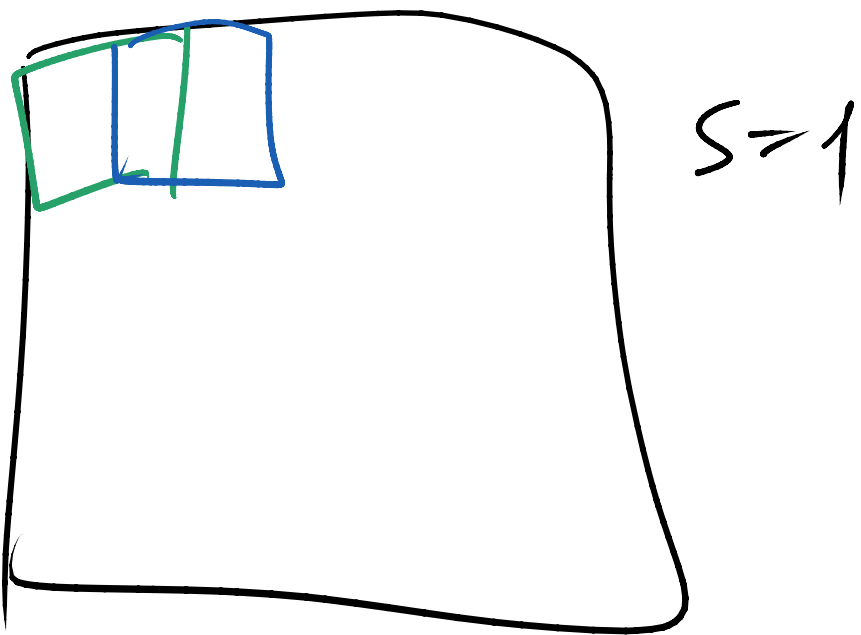
$$256 \rightarrow 128 \rightarrow 32 \rightarrow 10 \text{ (MLP)}$$

• kernel-size (k)

$$\text{kernel} = k^2, k_1 \times k_2$$

$$2 \times 2, 3 \times 3 \dots k_1 \times k_2$$

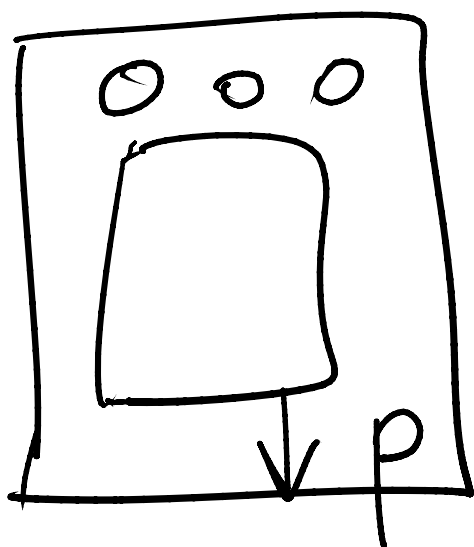
• stride



$$H \times H, k, s$$

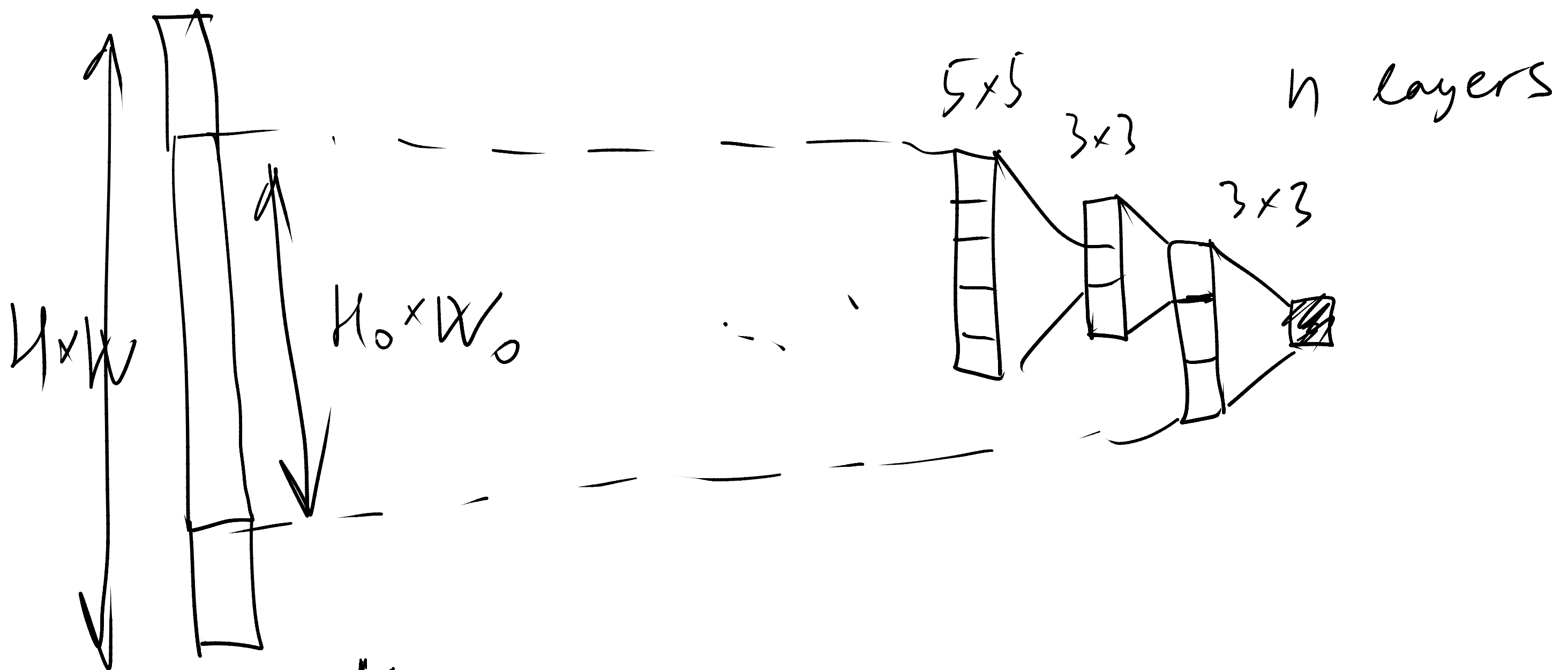
$$\left\lfloor \frac{H - k + 1}{s} \right\rfloor$$

• padding (p)



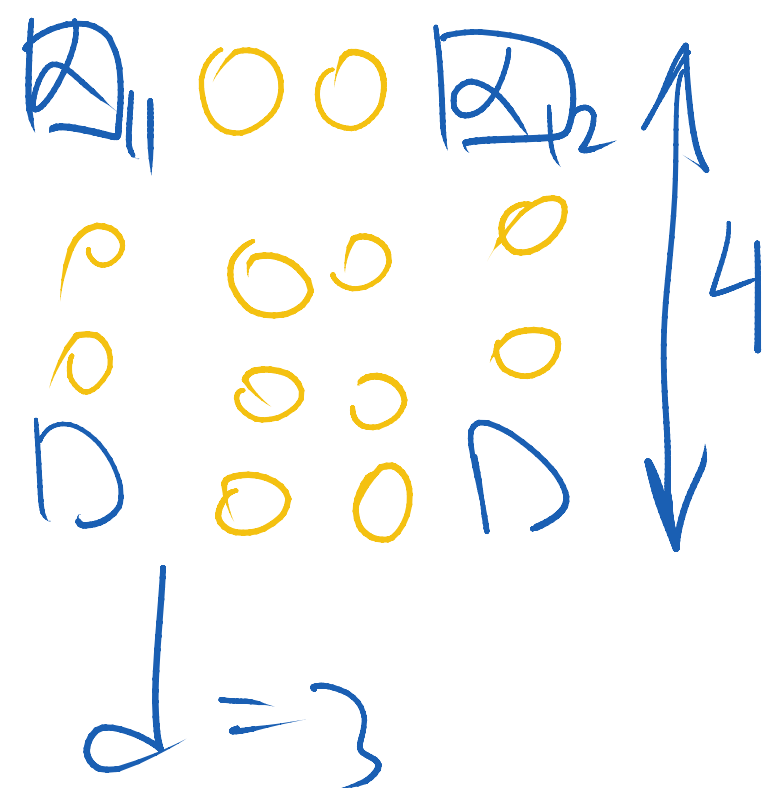
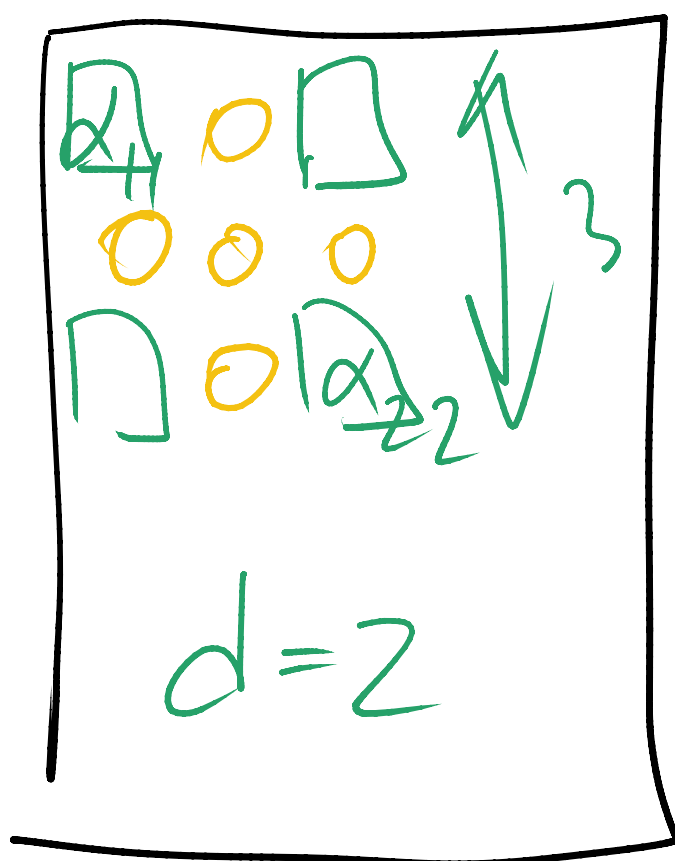
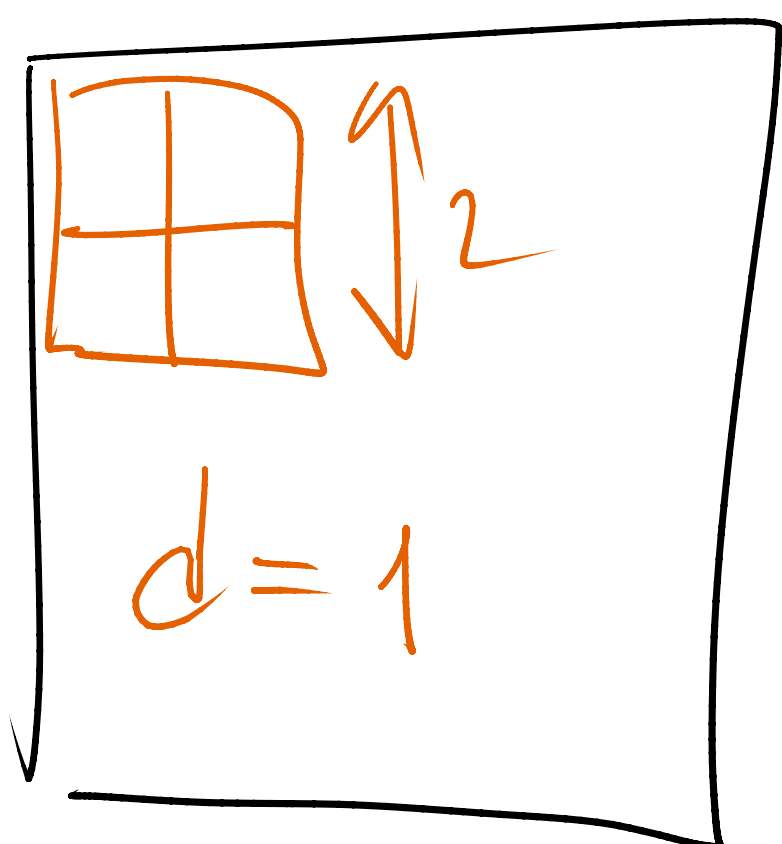
• dilation (d)

Receptive field



$H_0 \times W_0$ - receptive field

$$H_0 \leq H \quad W_0 \leq W$$

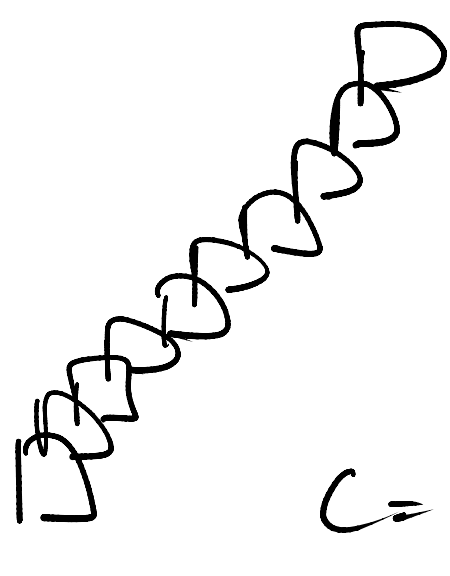
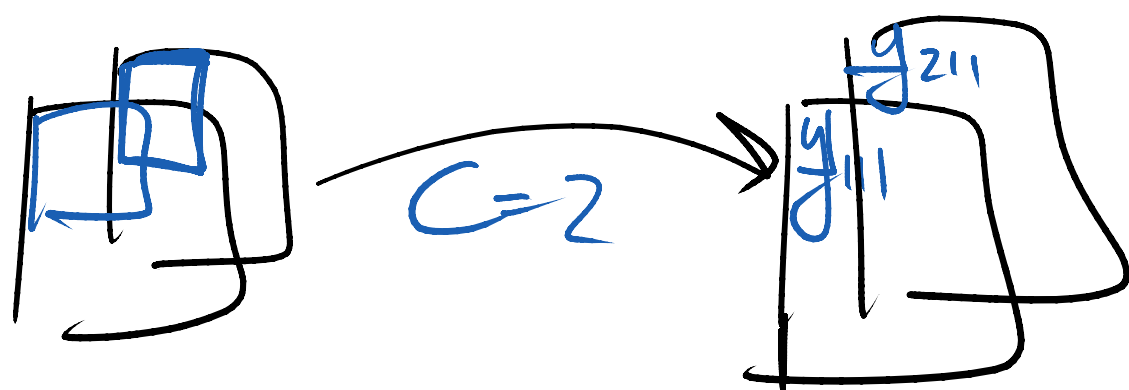


receptive field is larger

$H \times W \times C$ - numpy

Tensor

$C \times H \times W$ - pytorch



$h \times w$

$h < H$

$w < W$

$X: C_{in} \times H_{in} \times W_{in}$ $Y: C_{out} \times H_{out} \times W_{out}$

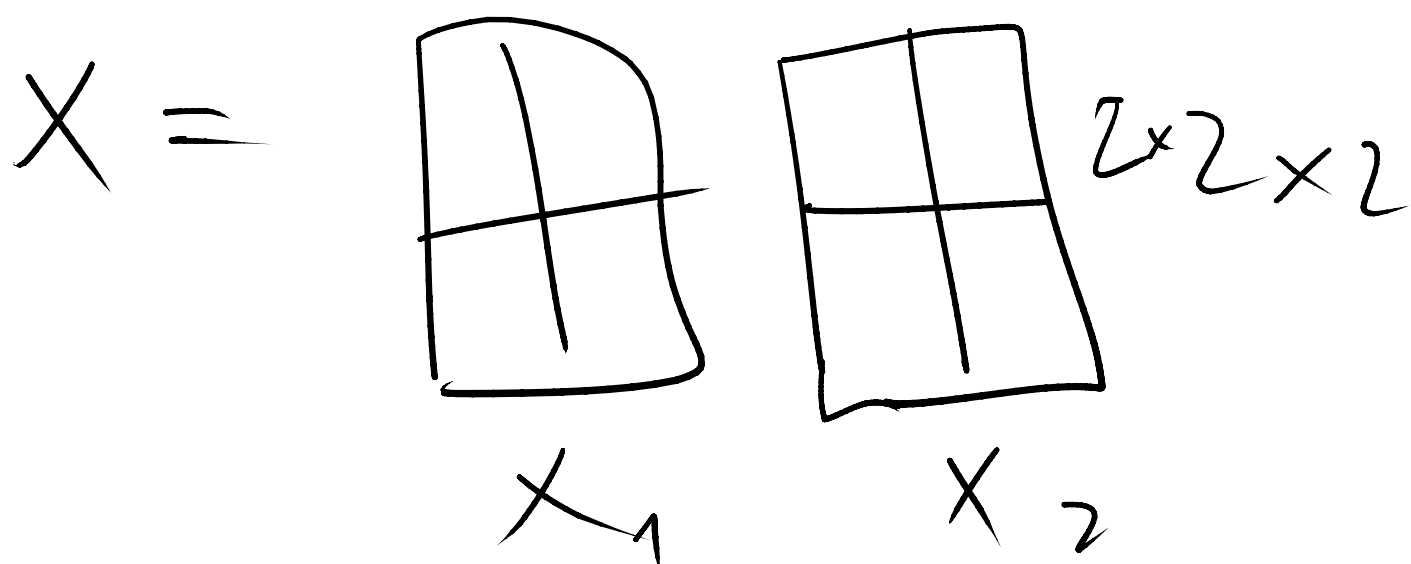
$\alpha: C_{in} \times C_{out} \times K_H \times K_W$

$$y_i = \sum_{j=1}^{C_{in}} x_j * \alpha_{ji}$$

$\uparrow$
2d conv

1×1 conv

$2 \times 3 \times 1 \times 1 - \alpha$



number = 6

$$\underset{\substack{\uparrow \\ \text{scalar}}}{\alpha_{1i}} \cdot \underset{\substack{\uparrow \\ 2 \times 2}}{x_1} + \alpha_{2i} \cdot x_2 \rightarrow y_i$$

y_1, y_2, y_3

$$y: 3 \times 2 \times 2$$

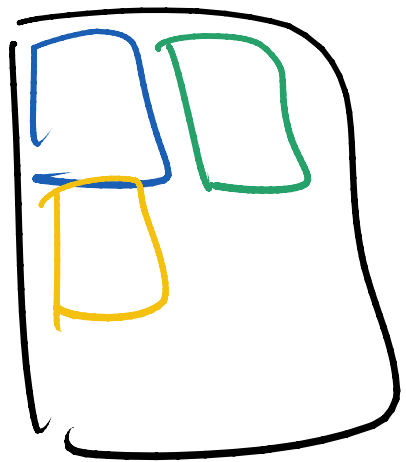
MLP

$$W \cdot x + b, \quad b \in \mathbb{R}^{d_n} \quad n\text{-layer}$$

CNN

$$y_i = \left(\sum_{j=1}^{C_{in}} x_j * \alpha_{ji} \right) + b_i, \quad b \in \mathbb{R}^{C_{out}}$$

• Average pooling

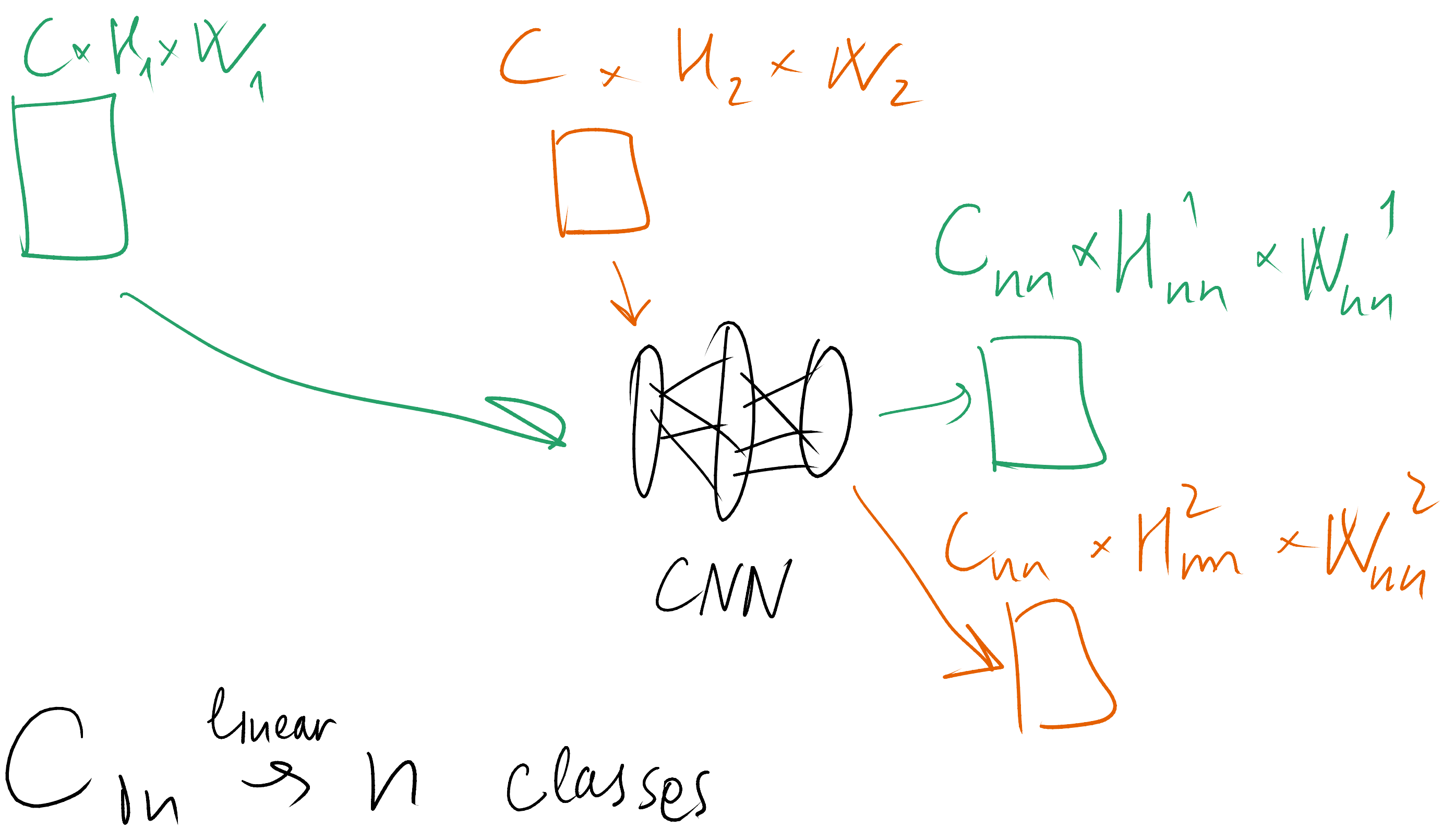


$$y = \frac{1}{k_1 \times k_2} \sum_{ij \in \square} x_{ij}$$

Stride = kernel size

• Adaptive pooling

Ex. Classification N classes



H_{out}, W_{out} — params of pooling
fixed tensor size

